

PROGRAMAÇÃO ORIENTADA A OBJETOS · UFG

shortlist.

Catálogo de filmes com recomendação por estratégias intercambiáveis — um caso de estudo de POO sobre dados reais.

AUTORAS

Giovanna Borges · Júlia Coité · Milena Oliveira

ORIENTAÇÃO

Prof. Dirson Campos · Instituto de Informática

0 PROBLEMA

Ordenar um catálogo por `sort()` é trivial.
Recomendar com critério, sobre dados reais, é um problema de modelagem.

25M

avaliações reais, agregadas a 58.958
filmes — o material que o modelo precisa
representar.

0

rede, modelo treinado ou credencial em
runtime — o produto consome só
snapshots estáticos.

P00

modelar a variedade dos filmes com
herança e polimorfismo, isolando decisões
em padrões formais.

As três telas

EXPLORAR O ACERVO

Filtre por gênero, categoria, ano ou título.

Buscar

Título — ex. Matrix, Cidade de Deus, Parasita...

Gênero

Todos os gêneros

Categoria

Todas as categorias

A partir do ano

1874

58.958 filme(s) · página 1/1180

1 - +

Título	Ano	Categoria	Nota	Nota ajustada	Votos
Planet Earth II	2016	Aclamado	4.48	4.44	1124
Planet Earth	2006	Blockbuster	4.46	4.44	1747
Shawshank Redemption, The	1994	Blockbuster	4.41	4.41	81482

01 Acervo

Navegação pela coleção encapsulada, somente leitura.

DISTRIBUIÇÃO DE POPULARIDADE

Os cortes que viram subclasse saem daqui — medidos, não inventados.

NOTA MÉDIA POR CATEGORIA

FILMES POR CATEGORIA

O deslatch do threshold - visto de cima

02 Estatísticas

Métricas do acervo calculadas pelo Analisador.

COMO VOCÊ QUER DESCOBRIR?

Quatro estratégias intercambiáveis (Strategy).

Estratégia

Similar (porque você gostou de X)

Quantos

5

Filme base

Scott Pilgrim vs. the World (2010)

Título	Ano	Categoria	Nota	Nota ajustada	Votos
Kick-Ass	2010	Blockbuster	3.61	3.61	11195
Zombieland	2009	Blockbuster	3.76	3.76	13628
Hot Fuzz	2007	Blockbuster	3.85	3.85	14356
Watchmen	2009	Blockbuster	3.71	3.71	11981
Shaun of the Dead	2004	Blockbuster	3.88	3.88	19558

03 Recomendações

Estratégia escolhida em runtime pela interface.

A TESE

As subclasses de **Filme** não foram inventadas a priori — elas **emergem da distribuição real** dos 58.958 filmes.

Mediram-se os percentis de `count`, `avg_rating` e `year`. Os cortes naturais da distribuição definem cada categoria — cada limiar é justificado por um número observado, não por opinião.

DA DISTRIBUIÇÃO ÀS CLASSES

Categorias que emergem do dado

SUBCLASSE	CRITÉRIO	~FILMES
Classico	antigo e ainda assistido — $\text{year} \leq 1970 \wedge \text{count} \geq 37$	1.9k
FilmeCult	nicho bem avaliado — $\text{count} \in [6,37] \wedge \text{avg_rating} \geq 3.9$	474
Blockbuster	topo 5% de popularidade — $\text{count} \geq \text{p95} \text{ (1508)}$	2.7k
FilmeAclamado	nota alta com votos — $\text{avg_rating} \geq 4.0 \wedge \text{count} \geq 6$	121
Filme (base)	fallback — tudo que não se encaixa acima	53.7k

HERANÇA

A hierarquia Filme



POLIMORFISMO

Um contrato, muitos scores

A interface chama o mesmo método; cada subclasse responde do seu jeito. A linguagem despacha em runtime — sem `if` de tipo na interface.

```
// uma única chamada  
  
ranking = sorted(  
    filmes,  
    key=lambda f: f.score()  
)  
  
cada f calcula seu próprio score
```

Classico

pondera longevidade e permanência no tempo

FilmeCult

premia nota alta dentro de um nicho pequeno

Blockbuster

escala o peso pela popularidade massiva

FilmeAclamado

prioriza a avaliação crítica acima do volume

ENCAPSULAMENTO

Cada objeto esconde seus internos

Catalogo

Coleção encapsulada, somente leitura.
Expõe consulta, esconde a estrutura interna.

Analizador

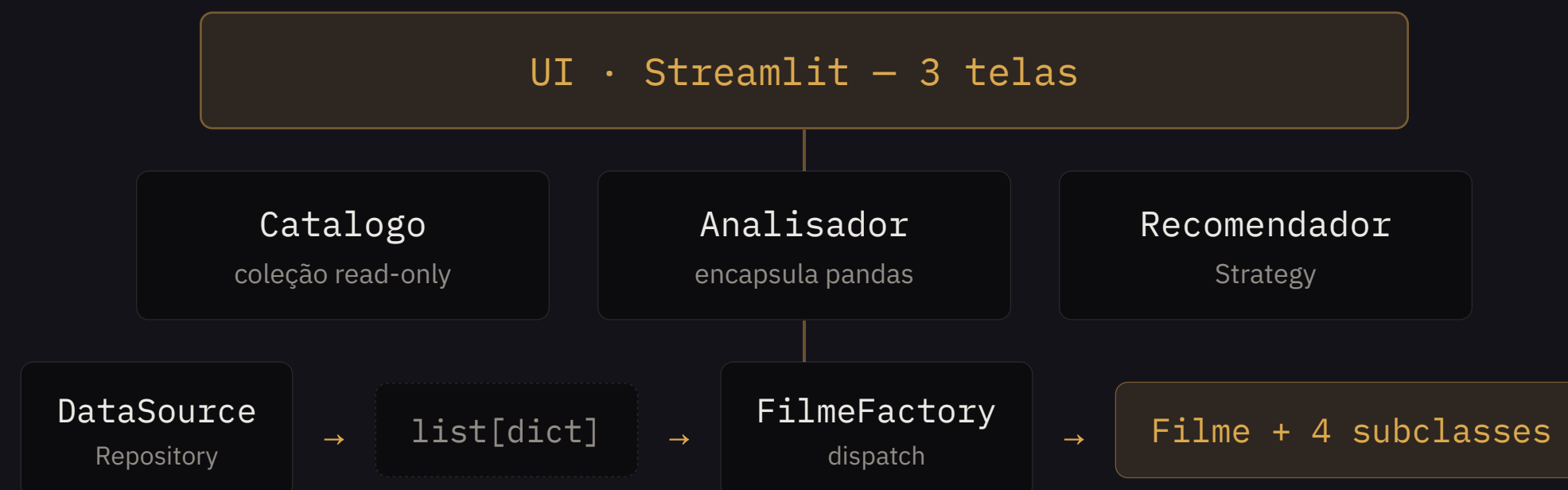
Encapsula **pandas** em 4 métricas. A UI vê números, nunca DataFrames.

Recomendador

Esconde o algoritmo atrás de um contrato único — base do padrão Strategy.

VISÃO GERAL

Arquitetura em camadas



Dois padrões formais isolam o que mais muda

Cada padrão protege uma decisão volátil atrás de um contrato estável — a UI nunca sabe qual implementação está usando.

PADRÃO 01

Strategy

Encapsula qual algoritmo recomenda. Quatro estratégias intercambiáveis sob o mesmo contrato.

PADRÃO 02

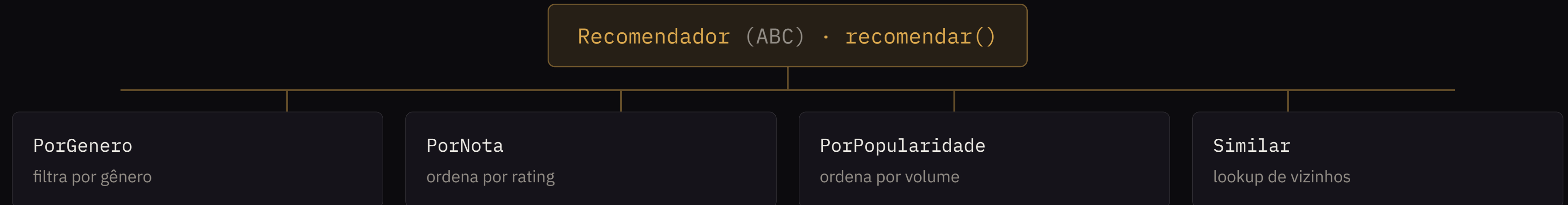
Repository

Encapsula de onde vêm os dados. CSV ou JSON sem que o domínio perceba a diferença.

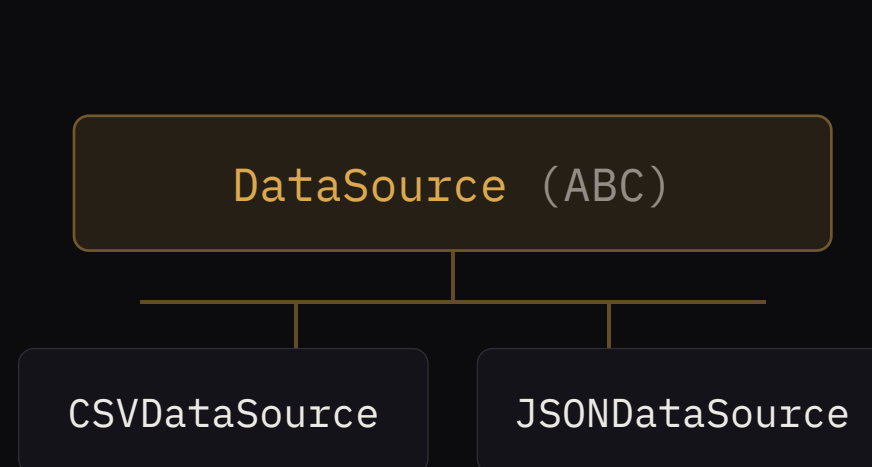
+ FilmeFactory — despacho dict → subclasse, tratado como construtor, não como Factory GoF.

Strategy: recomendação intercambiável

Algoritmos diferentes — filtro, ordenação, lookup de vizinhos — sob o mesmo contrato. A interface troca a estratégia sem saber qual é.



Repository: a fonte de dados



→

`list[dict]`

Devolve `list[dict]`, nunca `list[Filme]` — o repositório não conhece as subclasses. Trocar CSV por JSON não toca uma linha do domínio.

DA CAMADA DE DADOS AO DOMÍNIO

FilmeFactory: o despacho

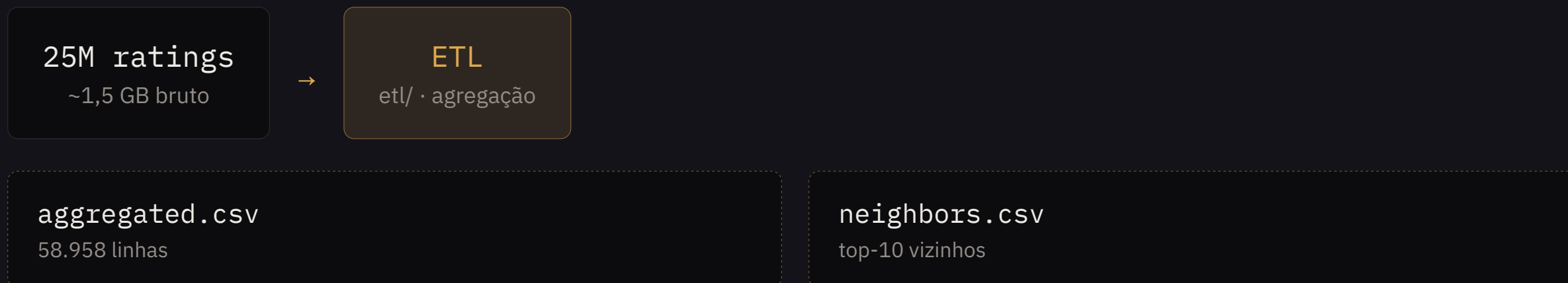
Guard clauses avaliadas por precedência. A ordem é contrato — peneiras sobrepostas que particionam o eixo de popularidade. Tratada como construtor com dispatch, não como Factory GoF.

1	Classico	$\text{year} \leq 1970 \wedge \text{count} \geq 37$
2	FilmeCult	$\text{count} \in [6,37] \wedge \text{avg_rating} \geq 3.9$
3	Blockbuster	$\text{count} \geq \text{p95 (1508)}$
4	FilmeAcclamado	$\text{avg_rating} \geq 4.0 \wedge \text{count} \geq 6$
5	Filme (base)	fallback

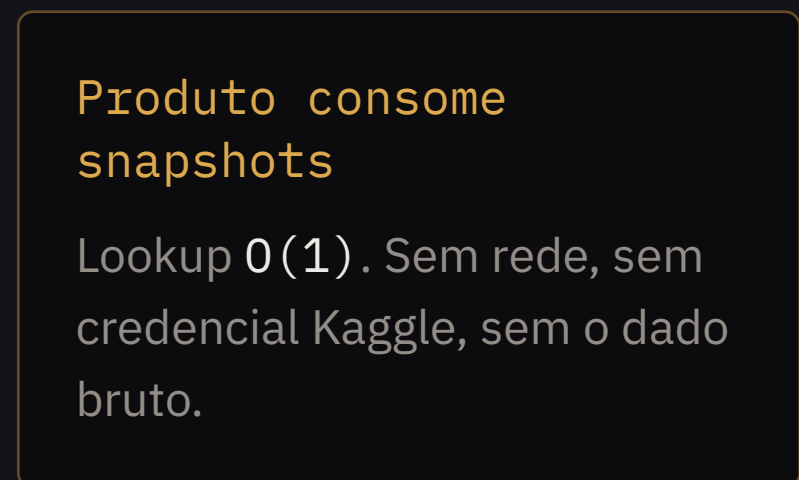
DADOS

Build-time vs runtime

BUILD-TIME · OFFLINE · RODA 1×



RUNTIME · O PRODUTO



ETL é fase, não produto.

ALÉM DO SORT()

Recomendação com critério

Rating bayesiano

Shrinkage que puxa a média de poucos votos rumo ao prior global, proporcional à evidência. Um filme com 1 voto e nota 5.0 não lidera o ranking.

Collaborative filtering

Similaridade de cosseno sobre co-avaliações, computada offline no ETL. Determinístico, sem modelo treinado — cosseno é álgebra linear, não ML.

QUALIDADE

Testes orientados ao núcleo

TDD seletivo no core de POO, com `parametrize` exercitando `Strategy` e `Repository` diretamente.

COBERTO · CORE POO

domínio · factory · datasource

catalogo · recomendador · analisador

FORA DE ESCOPO

UI — consumo

ETL — fase

CONCLUSÃO

O polimorfismo é o reflexo, em código, de uma estrutura que o **dado já tinha**.

Herança e polimorfismo traduzem a distribuição real em uma hierarquia de filmes.

Strategy e Repository isolam as decisões — algoritmo e fonte trocam sem tocar o domínio.

Build-time vs runtime mantém o produto leve: só snapshots, sem rede.

Demo ao vivo movie-shortlist.streamlit.app